

MCC Admin Portal — AI Outreach Engine

Replit Agent Build Instructions

PROJECT CONTEXT

This is a new feature module to be added to the existing **My Car Concierge (MCC)** platform admin portal. The platform is built with **React + Supabase**. This module adds a **continuously running, semi-autonomous AI-powered outreach engine** that discovers, scores, and populates a live opportunity pipeline with prospective **members, providers,** and **investors**, then drafts personalized outreach messages and queues them for admin approval.

The engine runs on an always-on loop — autonomously discovering new leads, generating drafts, and feeding the opportunity pipeline — but the admin retains full control via a **global Pause/Resume toggle** that halts all discovery and drafting activity immediately.

Do not scaffold a new app from scratch. This is a feature module. Generate modular, importable React components and Supabase-connected backend logic that can be dropped into the existing codebase.

CRITICAL: The outreach engine must be fully integrated with the existing MCC CRM.

The existing admin portal already manages members, providers, pending applications, support tickets, disputes, analytics, and payments — all built around the `profiles` table. The outreach engine is not a siloed tool. It reads from and writes back to the live CRM so that converted leads automatically become real platform users, and existing CRM contacts can be targeted for re-engagement campaigns without duplication.

TECH STACK

Layer	Technology
Frontend	React (existing app)
Backend/DB	Supabase (PostgreSQL + Edge Functions)

Realtime updates	Supabase Realtime (WebSocket subscriptions)
AI (message drafting + scoring)	Anthropic Claude API (<code>claude-sonnet-4-6</code>)
Email delivery	Resend API
SMS delivery	Twilio API
Scheduling	Supabase <code>pg_cron</code> + <code>pg_net</code> + Edge Functions
Lead sourcing (provider)	Google Places API

ENVIRONMENT VARIABLES REQUIRED

Add these to the Replit Secrets panel:

```
ANTHROPIC_API_KEY=  
RESEND_API_KEY=  
TWILIO_ACCOUNT_SID=  
TWILIO_AUTH_TOKEN=  
TWILIO_PHONE_NUMBER=  
GOOGLE_PLACES_API_KEY=  
SUPABASE_URL=  
SUPABASE_SERVICE_ROLE_KEY=  
SUPABASE_ANON_KEY=
```

DATABASE SCHEMA

Run the following SQL in the Supabase SQL editor to create all required tables.

```
-- LEADS TABLE: stores all prospective contacts  
CREATE TABLE outreach_leads (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  type TEXT NOT NULL CHECK (type IN ('member', 'provider', 'investor')),  
  name TEXT NOT NULL,  
  email TEXT,  
  phone TEXT,  
  company TEXT,  
  location TEXT,  
  source TEXT, -- 'manual', 'google_places', 'csv_import', 'organic'  
  status TEXT DEFAULT 'new' CHECK (status IN ('new', 'queued', 'contacted', 'resp
```

```

    notes TEXT,
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- OUTREACH MESSAGES TABLE: AI-drafted messages pending approval or sent
CREATE TABLE outreach_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    lead_id UUID REFERENCES outreach_leads(id) ON DELETE CASCADE,
    channel TEXT NOT NULL CHECK (channel IN ('email', 'sms')),
    sequence_step INTEGER DEFAULT 1,
    subject TEXT,
    body TEXT NOT NULL,
    status TEXT DEFAULT 'draft' CHECK (status IN ('draft', 'approved', 'sent', 'failed')),
    approved_by TEXT,
    approved_at TIMESTAMPTZ,
    sent_at TIMESTAMPTZ,
    opened_at TIMESTAMPTZ,
    clicked_at TIMESTAMPTZ,
    resend_message_id TEXT,
    twilio_message_sid TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- OUTREACH CAMPAIGNS TABLE: named campaigns grouping leads + rules
CREATE TABLE outreach_campaigns (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    target_type TEXT NOT NULL CHECK (target_type IN ('member', 'provider', 'investor')),
    channel TEXT NOT NULL CHECK (channel IN ('email', 'sms', 'both')),
    auto_send_followups BOOLEAN DEFAULT FALSE,
    first_touch_requires_approval BOOLEAN DEFAULT TRUE,
    status TEXT DEFAULT 'active' CHECK (status IN ('active', 'paused', 'archived')),
    message_template TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- CAMPAIGN LEADS JUNCTION
CREATE TABLE campaign_leads (
    campaign_id UUID REFERENCES outreach_campaigns(id) ON DELETE CASCADE,
    lead_id UUID REFERENCES outreach_leads(id) ON DELETE CASCADE,
    PRIMARY KEY (campaign_id, lead_id)
);

-- ACTIVITY LOG
CREATE TABLE outreach_activity_log (

```

```

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    lead_id UUID REFERENCES outreach_leads(id),
    message_id UUID REFERENCES outreach_messages(id),
    event_type TEXT NOT NULL, -- 'drafted', 'approved', 'sent', 'opened', 'clicked'
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- ENGINE STATE TABLE: global on/off control + configuration

```

CREATE TABLE engine_state (
    id INTEGER PRIMARY KEY DEFAULT 1 CHECK (id = 1), -- singleton row
    is_running BOOLEAN DEFAULT TRUE,
    paused_at TIMESTAMPTZ,
    paused_by TEXT,
    pause_reason TEXT,
    discovery_interval_minutes INTEGER DEFAULT 30, -- how often to scan for new lea
    max_drafts_per_cycle INTEGER DEFAULT 20,      -- cap on AI drafts per run
    target_cities TEXT[] DEFAULT ARRAY['East Rutherford, NJ', 'Newark, NJ', 'Jersey
    search_radius_meters INTEGER DEFAULT 15000,
    last_discovery_run TIMESTAMPTZ,
    last_draft_run TIMESTAMPTZ,
    total_leads_discovered INTEGER DEFAULT 0,
    total_messages_drafted INTEGER DEFAULT 0,
    total_messages_sent INTEGER DEFAULT 0,
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Insert the singleton row with defaults

```

INSERT INTO engine_state (id) VALUES (1);

```

-- OPPORTUNITY PIPELINE TABLE: scored, enriched view of leads ready for outreach

```

CREATE TABLE opportunity_pipeline (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    lead_id UUID REFERENCES outreach_leads(id) ON DELETE CASCADE UNIQUE,
    opportunity_score INTEGER DEFAULT 0 CHECK (opportunity_score BETWEEN 0 AND 100)
    score_rationale TEXT,
    priority TEXT DEFAULT 'medium' CHECK (priority IN ('high', 'medium', 'low')),
    recommended_channel TEXT DEFAULT 'email' CHECK (recommended_channel IN ('email'
    ai_notes TEXT,
    stage TEXT DEFAULT 'new' CHECK (stage IN ('new', 'draft_ready', 'message_queued
    added_at TIMESTAMPTZ DEFAULT NOW(),
    last_action_at TIMESTAMPTZ
);

```

-- LEADS TABLE: stores all prospective contacts

```

CREATE TABLE outreach_leads (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

type TEXT NOT NULL CHECK (type IN ('member', 'provider', 'investor')),
name TEXT NOT NULL,
email TEXT,
phone TEXT,
company TEXT,
location TEXT,
source TEXT, -- 'manual', 'google_places', 'csv_import', 'organic', 'continuous
status TEXT DEFAULT 'new' CHECK (status IN ('new', 'queued', 'contacted', 'resp
notes TEXT,
metadata JSONB DEFAULT '{}',
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- OUTREACH MESSAGES TABLE: AI-drafted messages pending approval or sent
CREATE TABLE outreach_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id UUID REFERENCES outreach_leads(id) ON DELETE CASCADE,
  channel TEXT NOT NULL CHECK (channel IN ('email', 'sms')),
  sequence_step INTEGER DEFAULT 1,
  subject TEXT,
  body TEXT NOT NULL,
  status TEXT DEFAULT 'draft' CHECK (status IN ('draft', 'approved', 'sent', 'fai
approved_by TEXT,
approved_at TIMESTAMPTZ,
sent_at TIMESTAMPTZ,
opened_at TIMESTAMPTZ,
clicked_at TIMESTAMPTZ,
resend_message_id TEXT,
twilio_message_sid TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- OUTREACH CAMPAIGNS TABLE: named campaigns grouping leads + rules
CREATE TABLE outreach_campaigns (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  target_type TEXT NOT NULL CHECK (target_type IN ('member', 'provider', 'investo
channel TEXT NOT NULL CHECK (channel IN ('email', 'sms', 'both')),
auto_send_followups BOOLEAN DEFAULT FALSE,
first_touch_requires_approval BOOLEAN DEFAULT TRUE,
status TEXT DEFAULT 'active' CHECK (status IN ('active', 'paused', 'archived'))
message_template TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- CAMPAIGN LEADS JUNCTION

```

```

CREATE TABLE campaign_leads (
  campaign_id UUID REFERENCES outreach_campaigns(id) ON DELETE CASCADE,
  lead_id UUID REFERENCES outreach_leads(id) ON DELETE CASCADE,
  PRIMARY KEY (campaign_id, lead_id)
);

-- ACTIVITY LOG
CREATE TABLE outreach_activity_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id UUID REFERENCES outreach_leads(id),
  message_id UUID REFERENCES outreach_messages(id),
  event_type TEXT NOT NULL, -- 'discovered', 'scored', 'drafted', 'approved', 'se
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Enable RLS (customize policies to match your auth setup)
ALTER TABLE engine_state ENABLE ROW LEVEL SECURITY;
ALTER TABLE opportunity_pipeline ENABLE ROW LEVEL SECURITY;
ALTER TABLE outreach_leads ENABLE ROW LEVEL SECURITY;
ALTER TABLE outreach_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE outreach_campaigns ENABLE ROW LEVEL SECURITY;
ALTER TABLE campaign_leads ENABLE ROW LEVEL SECURITY;
ALTER TABLE outreach_activity_log ENABLE ROW LEVEL SECURITY;

```

CRM INTEGRATION LAYER

The outreach engine connects to the existing MCC CRM at four key touchpoints. These integrations must be implemented before or alongside the Edge Functions.

1. Schema Bridge — Linking Outreach Leads to CRM Profiles

Add the following columns to the **existing** `outreach_leads` table to create a two-way link with the `profiles` table. Run this SQL on the existing database — do not recreate the table:

```

-- Link outreach leads to existing CRM profiles (nullable — only set when convert
ALTER TABLE outreach_leads
  ADD COLUMN IF NOT EXISTS crm_profile_id UUID REFERENCES profiles(id) ON DELETE
  ADD COLUMN IF NOT EXISTS crm_sync_status TEXT DEFAULT 'unlinked'
  CHECK (crm_sync_status IN ('unlinked', 'linked', 'converted', 'duplicate'));

-- Index for fast lookups

```

```

CREATE INDEX IF NOT EXISTS idx_outreach_leads_crm_profile ON outreach_leads(crm_p
CREATE INDEX IF NOT EXISTS idx_outreach_leads_email ON outreach_leads(email);

-- Add outreach context to the existing profiles table so the CRM shows outreach
ALTER TABLE profiles
  ADD COLUMN IF NOT EXISTS outreach_lead_id UUID REFERENCES outreach_leads(id) ON
  ADD COLUMN IF NOT EXISTS outreach_source TEXT,      -- where they came from ('go
  ADD COLUMN IF NOT EXISTS outreach_converted_at TIMESTAMPTZ;

-- Add outreach tab to existing provider_applications for re-engagement tracking
ALTER TABLE provider_applications
  ADD COLUMN IF NOT EXISTS outreach_lead_id UUID REFERENCES outreach_leads(id) ON

```

2. Deduplication — Prevent Targeting Existing CRM Users

Before any lead is added to `outreach_leads` or the opportunity pipeline, the system must check whether the contact already exists in `profiles`. This runs in the `import-provider-leads` function and any manual lead import.

```

-- Helper function: check if an email or phone already exists in the CRM
CREATE OR REPLACE FUNCTION check_crm_duplicate(p_email TEXT, p_phone TEXT)
RETURNS TABLE (exists_in_crm BOOLEAN, profile_id UUID, profile_role TEXT)
LANGUAGE sql AS $$
  SELECT
    TRUE,
    id,
    role
  FROM profiles
  WHERE
    (p_email IS NOT NULL AND email = p_email)
    OR (p_phone IS NOT NULL AND phone = p_phone)
  LIMIT 1;
$$;

```

In every import and discovery flow, call this function first. If a match is found:

- If `role = 'provider'` or `role = 'member'`: mark the lead as `crm_sync_status = 'duplicate'`, set `crm_profile_id`, and **skip outreach** — they're already in the platform
 - If `role = 'pending_provider'`: mark as `crm_sync_status = 'linked'` — they're in the application pipeline; the outreach engine should **not contact them independently** but surface them in the CRM view as a warm lead with pending application status
-

3. Conversion Trigger — Lead → CRM Profile

When an outreach lead converts (signs up for MCC after being contacted), this Edge Function links the accounts and updates both tables.

```
// supabase/functions/convert-lead-to-profile/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

serve(async (req) => {
  const { lead_id, profile_id } = await req.json();
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  // Fetch both records
  const { data: lead } = await supabase.from("outreach_leads").select("*").eq("id", lead_id);
  const { data: profile } = await supabase.from("profiles").select("*").eq("id", profile_id);

  if (!lead || !profile) {
    return new Response(JSON.stringify({ error: "Lead or profile not found" }), {
      status: 400
    });
  }

  // Link the lead to the profile
  await supabase.from("outreach_leads").update({
    status: "converted",
    crm_profile_id: profile_id,
    crm_sync_status: "converted"
  }).eq("id", lead_id);

  // Tag the profile with outreach attribution
  await supabase.from("profiles").update({
    outreach_lead_id: lead_id,
    outreach_source: lead.source,
    outreach_converted_at: new Date().toISOString()
  }).eq("id", profile_id);

  // Update pipeline stage
  await supabase.from("opportunity_pipeline").update({
    stage: "converted",
    last_action_at: new Date().toISOString()
  }).eq("lead_id", lead_id);

  // Log the conversion
  await supabase.from("outreach_activity_log").insert({
    lead_id,
    event_type: "converted",
  });
});
```

```

    metadata: { profile_id, profile_role: profile.role }
  });

  // Update engine stats
  await supabase.rpc("increment_engine_stat", { stat: "total_conversions", amount

  return new Response(JSON.stringify({ success: true, profile_id, lead_id }), {
    headers: { "Content-Type": "application/json" }
  });
});
});

```

Auto-trigger: Add a Supabase Database Webhook that fires `convert-lead-to-profile` whenever a new row is inserted into `profiles` — passing the new `profile_id` — so the system automatically checks if a matching outreach lead exists by email or phone and links them:

```

-- Trigger to auto-detect conversions when new profiles are created
CREATE OR REPLACE FUNCTION auto_link_outreach_lead()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
  matched_lead_id UUID;
BEGIN
  -- Look for an outreach lead matching the new profile's email or phone
  SELECT id INTO matched_lead_id
  FROM outreach_leads
  WHERE
    crm_profile_id IS NULL
    AND crm_sync_status != 'duplicate'
    AND (
      (NEW.email IS NOT NULL AND email = NEW.email)
      OR (NEW.phone IS NOT NULL AND phone = NEW.phone)
    )
  LIMIT 1;

  IF matched_lead_id IS NOT NULL THEN
    -- Fire the conversion function asynchronously via pg_net
    PERFORM net.http_post(
      url := current_setting('app.supabase_url') || '/functions/v1/convert-lead-t
      headers := jsonb_build_object(
        'Authorization', 'Bearer ' || current_setting('app.service_role_key'),
        'Content-Type', 'application/json'
      ),
      body := jsonb_build_object('lead_id', matched_lead_id, 'profile_id', NEW.id
    );
  END IF;

```

```

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_auto_link_outreach_lead
  AFTER INSERT ON profiles
  FOR EACH ROW EXECUTE FUNCTION auto_link_outreach_lead();

```

4. CRM Re-Engagement — Target Existing Profiles

The engine can also reach out to **existing CRM users** for re-engagement — for example, members who signed up but never created a service request, or providers who applied but went inactive. These are surfaced as a special lead type pulled directly from `profiles`.

Add this Edge Function to identify and queue re-engagement leads:

```

// supabase/functions/sync-crm-reengagement/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

serve(async (req) => {
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  // Guard: check engine state
  const { data: state } = await supabase.from("engine_state").select("is_running")
  if (!state?.is_running) return new Response(JSON.stringify({ skipped: true }));

  const reengagementTargets: any[] = [];

  // Case 1: Members who signed up 7+ days ago but never created a service request
  const { data: dormantMembers } = await supabase.rpc("get_dormant_members");
  for (const m of (dormantMembers || [])) {
    reengagementTargets.push({
      type: "member",
      name: m.full_name || m.email,
      email: m.email,
      phone: m.phone,
      source: "crm_reengagement",
      crm_profile_id: m.id,
      crm_sync_status: "linked",
      notes: "Signed up but has not yet created a service request.",
    });
  }
});

```

```

        status: "new"
    });
}

// Case 2: Pending providers in the application pipeline with no activity for 5
const { data: stalledProviders } = await supabase.rpc("get_stalled_applications
for (const p of (stalledProviders || [])) {
    reengagementTargets.push({
        type: "provider",
        name: p.business_name || p.full_name,
        email: p.email,
        phone: p.phone,
        source: "crm_reengagement",
        crm_profile_id: p.id,
        crm_sync_status: "linked",
        notes: `Provider application stalled. Current status: ${p.application_status}
        status: "new"
    });
}

// Upsert into outreach_leads (skip if already exists with this crm_profile_id)
if (reengagementTargets.length > 0) {
    await supabase.from("outreach_leads").upsert(reengagementTargets, {
        onConflict: "crm_profile_id",
        ignoreDuplicates: true
    });
}

return new Response(JSON.stringify({ queued: reengagementTargets.length }), {
    headers: { "Content-Type": "application/json" }
});
});

```

Add these SQL helper functions:

```

-- Members who never created a service request
CREATE OR REPLACE FUNCTION get_dormant_members()
RETURNS TABLE (id UUID, full_name TEXT, email TEXT, phone TEXT)
LANGUAGE sql AS $$
    SELECT p.id, p.full_name, p.email, p.phone
    FROM profiles p
    WHERE p.role = 'member'
        AND p.created_at < NOW() - INTERVAL '7 days'
        AND NOT EXISTS (
            SELECT 1 FROM maintenance_packages mp WHERE mp.member_id = p.id
        )

```

```

        AND p.outreach_lead_id IS NULL -- not already in outreach
    LIMIT 100;
$$;

-- Providers whose applications have been idle for 5+ days
CREATE OR REPLACE FUNCTION get_stalled_applications()
RETURNS TABLE (id UUID, full_name TEXT, business_name TEXT, email TEXT, phone TEXT)
LANGUAGE sql AS $$
    SELECT
        p.id, p.full_name, p.business_name, p.email, p.phone,
        pa.status AS application_status
    FROM profiles p
    JOIN provider_applications pa ON pa.provider_id = p.id
    WHERE p.role = 'pending_provider'
        AND pa.updated_at < NOW() - INTERVAL '5 days'
        AND pa.status NOT IN ('rejected', 'approved')
        AND p.outreach_lead_id IS NULL
    LIMIT 50;
$$;

```

5. CRM Context Panel — Show Outreach History in Existing Admin Views

In the **existing** admin portal provider and member detail views, add an **Outreach History** collapsible panel that shows the outreach timeline for that contact without requiring the admin to leave the CRM view.

Create `OutreachHistoryPanel.jsx` — a lightweight embeddable component:

```

// src/admin/outreach/OutreachHistoryPanel.jsx
// Embed this inside the existing MemberDetailView and ProviderDetailView components

import { useEffect, useState } from "react";
import { supabase } from "../../supabaseClient";

export function OutreachHistoryPanel({ profileId }) {
    const [lead, setLead] = useState(null);
    const [messages, setMessages] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const load = async () => {
            // Find the outreach lead linked to this CRM profile
            const { data: leadData } = await supabase
                .from("outreach_leads")

```

```

        .select("")
        .eq("crm_profile_id", profileId)
        .maybeSingle();

    if (leadData) {
        setLead(leadData);
        const { data: msgs } = await supabase
            .from("outreach_messages")
            .select("")
            .eq("lead_id", leadData.id)
            .order("created_at", { ascending: false });
        setMessages(msgs || []);
    }
    setLoading(false);
};

load();
}, [profileId]);

if (loading) return <div className="outreach-panel loading">Loading outreach hi
if (!lead) return (
    <div className="outreach-panel empty">
        <span>No outreach history – this user joined organically or was not part of
    </div>
);

return (
    <div className="outreach-panel">
        <h4>Outreach History</h4>
        <div className="outreach-meta">
            <span>Source: <strong>{lead.source}</strong></span>
            <span>First contacted: <strong>{new Date(lead.created_at).toLocaleDateStr
            <span>Status: <strong>{lead.status}</strong></span>
        </div>
        <div className="outreach-messages">
            {messages.length === 0 && <p>No messages sent yet.</p>}
            {messages.map(msg => (
                <div key={msg.id} className={`outreach-message-row status-${msg.status}>
                    <span className="msg-channel">{msg.channel.toUpperCase()}</span>
                    <span className="msg-subject">{msg.subject || "(SMS)"}</span>
                    <span className="msg-status">{msg.status}</span>
                    <span className="msg-date">{new Date(msg.created_at).toLocaleDateStri
                </div>
            ))}
        </div>
    </div>
);
}

```

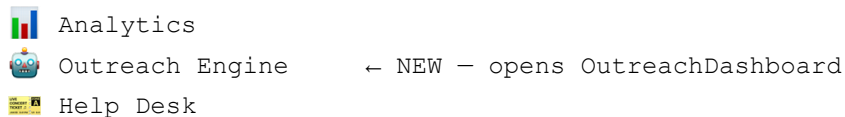
Wire this into the existing admin portal by importing and rendering

```
<OutreachHistoryPanel profileId={selectedProfile.id} /> inside:
```

- The existing **Provider Detail / Application Review** view
- The existing **Member Detail** view
- The existing **Help Desk ticket** view (so you can see if a ticket-submitter came through outreach)

6. Navigation — Embed Outreach Engine in Existing Admin Sidebar

The outreach engine should appear as a new section in the **existing admin sidebar**, not as a separate app. Add a new nav item between Analytics and Help Desk:



When clicked, it renders `<OutreachDashboard />` in the existing admin content area — the same pattern used by all other admin sections.

SUPABASE EDGE FUNCTIONS

Create the following Edge Functions in `/supabase/functions/`.

0. `engine-cycle` ← THE CORE CONTINUOUS LOOP

Purpose: This is the master orchestrator. It runs on a short cron interval, checks if the engine is running, and dispatches sub-tasks: lead discovery, scoring, and draft generation. All other functions are called by this one.

```
// supabase/functions/engine-cycle/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

serve(async (req) => {
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
```

```

    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

// --- GUARD: Check engine state ---
const { data: state } = await supabase
  .from("engine_state")
  .select("*")
  .eq("id", 1)
  .single();

if (!state || !state.is_running) {
  console.log("Engine is paused. Skipping cycle.");
  return new Response(JSON.stringify({ skipped: true, reason: "engine_paused" }
    headers: { "Content-Type": "application/json" }
  ));
}

const now = new Date();
const lastDiscovery = state.last_discovery_run ? new Date(state.last_discovery_
const minutesSinceDiscovery = lastDiscovery
  ? (now.getTime() - lastDiscovery.getTime()) / 60000
  : Infinity;

const baseUrl = Deno.env.get("SUPABASE_URL") + "/functions/v1";
const headers = {
  "Authorization": `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")}`,
  "Content-Type": "application/json"
};

const results: any = { cycle_at: now.toISOString() };

// --- STEP 1: Discover new leads (respects interval setting) ---
if (minutesSinceDiscovery >= state.discovery_interval_minutes) {
  for (const city of (state.target_cities || [])) {
    await fetch(`${baseUrl}/import-provider-leads`, {
      method: "POST", headers,
      body: JSON.stringify({ location: city, radius_meters: state.search_radius
    });
  }
  // Also sync CRM re-engagement targets
  await fetch(`${baseUrl}/sync-crm-reengagement`, { method: "POST", headers, bo
  await supabase.from("engine_state").update({ last_discovery_run: now.toISOStr
  results.discovery = "ran";
} else {
  results.discovery = "skipped (interval not reached)";
}

```

```

// --- STEP 2: Score unscored leads and add to pipeline ---
const { data: unscoredLeads } = await supabase
  .from("outreach_leads")
  .select("id, type, name, location, metadata, notes")
  .not("id", "in",
    supabase.from("opportunity_pipeline").select("lead_id")
  )
  .limit(50);

if (unscoredLeads && unscoredLeads.length > 0) {
  const scoreRes = await fetch(`${baseUrl}/score-leads`, {
    method: "POST", headers,
    body: JSON.stringify({ leads: unscoredLeads })
  });
  const scoreResult = await scoreRes.json();
  results.scored = scoreResult.scored ?? 0;
}

// --- STEP 3: Draft messages for high-priority pipeline leads with no draft yet
const { data: draftTargets } = await supabase
  .from("opportunity_pipeline")
  .select("lead_id, recommended_channel, outreach_leads(*)")
  .in("stage", ["new", "draft_ready"])
  .in("priority", ["high", "medium"])
  .limit(state.max_drafts_per_cycle || 20);

let drafted = 0;
for (const opp of (draftTargets || [])) {
  // Check engine state hasn't been paused mid-cycle
  const { data: freshState } = await supabase.from("engine_state").select("is_running")
  if (!freshState?.is_running) {
    console.log("Engine paused mid-cycle. Stopping draft loop.");
    break;
  }

  const lead = opp.outreach_leads;
  if (!lead?.email && !lead?.phone) continue;

  // Skip leads that are already active CRM users (duplicates)
  if (lead.crm_sync_status === "duplicate") continue;

  // Check no existing draft for this lead
  const { count } = await supabase
    .from("outreach_messages")
    .select("id", { count: "exact", head: true })
    .eq("lead_id", opp.lead_id)
    .in("status", ["draft", "approved", "sent"]);

```

```

    if (count && count > 0) continue;

    const draftRes = await fetch(`${baseUrl}/draft-outreach-message`, {
      method: "POST", headers,
      body: JSON.stringify({
        lead,
        channel: opp.recommended_channel || "email",
        sequence_step: 1
      })
    });
    const { subject, body } = await draftRes.json();

    await supabase.from("outreach_messages").insert({
      lead_id: opp.lead_id,
      channel: opp.recommended_channel || "email",
      subject,
      body,
      status: "draft"
    });

    await supabase.from("opportunity_pipeline")
      .update({ stage: "message_queued", last_action_at: new Date().toISOString() })
      .eq("lead_id", opp.lead_id);

    drafted++;
  }

  results.drafted = drafted;

  // --- Update engine stats ---
  await supabase.from("engine_state").update({
    last_draft_run: now.toISOString(),
    total_leads_discovered: supabase.rpc("increment", { x: unscoredLeads?.length }),
    total_messages_drafted: supabase.rpc("increment", { x: drafted }),
    updated_at: now.toISOString()
  }).eq("id", 1);

  return new Response(JSON.stringify({ success: true, ...results }), {
    headers: { "Content-Type": "application/json" }
  });
});

```

0b. score-leads

Purpose: Uses Claude to evaluate and score a batch of leads, then inserts them into the opportunity pipeline with a priority and recommended channel.

```
// supabase/functions/score-leads/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";
import Anthropic from "npm:@anthropic-ai/sdk";

const anthropic = new Anthropic({ apiKey: Deno.env.get("ANTHROPIC_API_KEY") });

serve(async (req) => {
  const { leads } = await req.json();
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  const prompt = `You are a lead qualification assistant for My Car Concierge (MC

Score each of the following leads from 0-100 based on how likely they are to conv

For providers (auto repair shops): high scores for shops with many reviews, good
For members (vehicle owners): high scores for leads with complete contact info.
For investors: high scores for leads with specific company/title context.

Respond ONLY with a valid JSON array, one object per lead, in this format:
[
  {
    "lead_id": "uuid",
    "opportunity_score": 75,
    "priority": "high",
    "recommended_channel": "email",
    "score_rationale": "Well-rated shop with 200+ reviews",
    "ai_notes": "Mention competitive customer volume in outreach"
  }
]

Leads to score:
${JSON.stringify(leads, null, 2)}`;

  const response = await anthropic.messages.create({
    model: "claude-sonnet-4-6",
    max_tokens: 2000,
    messages: [{ role: "user", content: prompt }]
  });
```

```

const text = response.content[0].type === "text" ? response.content[0].text : "
let scores = [];
try {
  const clean = text.replace(/```json|```/g, "").trim();
  scores = JSON.parse(clean);
} catch (e) {
  return new Response(JSON.stringify({ error: "Failed to parse scores", raw: te
}

const pipelineRows = scores.map((s: any) => ({
  lead_id: s.lead_id,
  opportunity_score: s.opportunity_score,
  priority: s.priority,
  recommended_channel: s.recommended_channel,
  score_rationale: s.score_rationale,
  ai_notes: s.ai_notes,
  stage: "draft_ready"
})));

const { error } = await supabase.from("opportunity_pipeline").upsert(pipelineRo
  onConflict: "lead_id",
  ignoreDuplicates: false
});

return new Response(JSON.stringify({ scored: scores.length, error }), {
  headers: { "Content-Type": "application/json" }
});
});

```

0c. toggle-engine

Purpose: Pause or resume the engine. Called by the admin UI pause toggle.

```

// supabase/functions/toggle-engine/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

serve(async (req) => {
  const { is_running, paused_by, pause_reason } = await req.json();
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  const update: any = {

```

```

    is_running,
    updated_at: new Date().toISOString()
  };

  if (!is_running) {
    update.paused_at = new Date().toISOString();
    update.paused_by = paused_by || "admin";
    update.pause_reason = pause_reason || null;
  } else {
    update.paused_at = null;
    update.paused_by = null;
    update.pause_reason = null;
  }

  const { error } = await supabase.from("engine_state").update(update).eq("id", 1

  // Log the event
  await supabase.from("outreach_activity_log").insert({
    event_type: is_running ? "engine_resumed" : "engine_paused",
    metadata: { paused_by, pause_reason }
  });

  return new Response(JSON.stringify({ success: !error, is_running, error }), {
    headers: { "Content-Type": "application/json" }
  });
});

```

1. draft-outreach-message

Purpose: Call Anthropic API to generate a personalized outreach message for a lead.

```

// supabase/functions/draft-outreach-message/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import Anthropic from "npm:@anthropic-ai/sdk";

const anthropic = new Anthropic({ apiKey: Deno.env.get("ANTHROPIC_API_KEY") });

serve(async (req) => {
  const { lead, channel, sequence_step, campaign } = await req.json();

  const audienceContext = {
    member: "a vehicle owner looking for reliable, transparent auto repair servic
    provider: "an auto repair shop or mechanic that could benefit from a new sour
    investor: "a potential investor or strategic partner evaluating the My Car Co
  };

```

```

const channelInstruction = channel === "sms"
  ? "Write a brief SMS message under 160 characters. Be direct, friendly, and i
  : "Write a professional email. Include a subject line on the first line in th

const followUpNote = sequence_step > 1
  ? `This is follow-up message #${sequence_step}. Reference that we reached out
  : "This is the first outreach. Introduce My Car Concierge warmly and clearly.

const reengagementNote = lead.source === "crm_reengagement"
  ? "IMPORTANT: This person is an existing MCC user. Do NOT pitch the platform
  : "";

const prompt = `You are a growth outreach assistant for My Car Concierge (MCC),

You are writing a ${channel} message to ${audienceContext[lead.type]}.

Lead details:
- Name: ${lead.name}
- Company/Location: ${lead.company || lead.location || "Not provided"}
- Notes: ${lead.notes || "None"}
- Source: ${lead.source}

Campaign context: ${campaign?.message_template || "Standard outreach"}

${reengagementNote}

Instructions:
- ${channelInstruction}
- ${followUpNote}
- Tone: warm, professional, concise
- Never be pushy or salesy
- For investors: do NOT make specific ROI promises or securities claims
- Always end with a clear next step (call, sign-up link, or reply to learn more)
- Use [FIRST_NAME] as a placeholder if personalizing the greeting
- Platform link placeholder: [MCC_LINK]

Write the message now:`;

const response = await anthropic.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 600,
  messages: [{ role: "user", content: prompt }]
});

const text = response.content[0].type === "text" ? response.content[0].text : "

```

```

let subject = null;
let body = text;

if (channel === "email") {
  const lines = text.split("\n");
  const subjectLine = lines.find(l => l.startsWith("Subject:"));
  if (subjectLine) {
    subject = subjectLine.replace("Subject:", "").trim();
    body = lines.slice(lines.indexOf(subjectLine) + 2).join("\n").trim();
  }
}

return new Response(JSON.stringify({ subject, body }), {
  headers: { "Content-Type": "application/json" }
});
});

```

2. send-outreach-message

Purpose: Send an approved message via Resend (email) or Twilio (SMS).

```

// supabase/functions/send-outreach-message/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

serve(async (req) => {
  const { message_id } = await req.json();
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  // Fetch message + lead
  const { data: msg } = await supabase
    .from("outreach_messages")
    .select("*, outreach_leads(*)")
    .eq("id", message_id)
    .single();

  if (!msg || msg.status !== "approved") {
    return new Response(JSON.stringify({ error: "Message not found or not approve" }));
  }

  const lead = msg.outreach_leads;
  const firstName = lead.name.split(" ")[0];

```

```

const body = msg.body
  .replace("[FIRST_NAME]", firstName)
  .replace("[MCC_LINK]", "https://mycarconciiergepro.com")
  .replace("[LINK]", "https://mycarconciiergepro.com");

let externalId = null;
let error = null;

if (msg.channel === "email" && lead.email) {
  // Send via Resend
  const res = await fetch("https://api.resend.com/emails", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${Deno.env.get("RESEND_API_KEY")}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      from: "Jordan Zanetis <jordan@mycarconciiergepro.com>",
      to: [lead.email],
      subject: msg.subject || "My Car Concierge - Let's Connect",
      text: body
    })
  });
  const result = await res.json();
  externalId = result.id;
  if (!res.ok) error = result;
} else if (msg.channel === "sms" && lead.phone) {
  // Send via Twilio
  const accountSid = Deno.env.get("TWILIO_ACCOUNT_SID");
  const authToken = Deno.env.get("TWILIO_AUTH_TOKEN");
  const fromPhone = Deno.env.get("TWILIO_PHONE_NUMBER");

  const res = await fetch(
    `https://api.twilio.com/2010-04-01/Accounts/${accountSid}/Messages.json`,
    {
      method: "POST",
      headers: {
        "Authorization": "Basic " + btoa(`${accountSid}:${authToken}`),
        "Content-Type": "application/x-www-form-urlencoded"
      },
      body: new URLSearchParams({ From: fromPhone!, To: lead.phone, Body: body })
    }
  );
  const result = await res.json();
  externalId = result.sid;
  if (!res.ok) error = result;
}

```

```

    }

    // Update message record
    await supabase.from("outreach_messages").update({
      status: error ? "failed" : "sent",
      sent_at: error ? null : new Date().toISOString(),
      resend_message_id: msg.channel === "email" ? externalId : null,
      twilio_message_sid: msg.channel === "sms" ? externalId : null
    }).eq("id", message_id);

    // Log activity
    await supabase.from("outreach_activity_log").insert({
      lead_id: lead.id,
      message_id: msg.id,
      event_type: error ? "send_failed" : "sent",
      metadata: error ? { error } : { external_id: externalId }
    });

    // Update lead status
    if (!error) {
      await supabase.from("outreach_leads").update({ status: "contacted" }).eq("id",
    }

    return new Response(JSON.stringify({ success: !error, error }), {
      headers: { "Content-Type": "application/json" }
    });
  });
});

```

3. import-provider-leads

Purpose: Pull nearby auto repair shops from Google Places and insert them as provider leads.

```

// supabase/functions/import-provider-leads/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

serve(async (req) => {
  const { location, radius_meters = 10000 } = await req.json();
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );
  const apiKey = Deno.env.get("GOOGLE_PLACES_API_KEY");

```

```

// Geocode location first
const geoRes = await fetch(
  `https://maps.googleapis.com/maps/api/geocode/json?address=${encodeURIComponent(
  )};
const geoData = await geoRes.json();
const coords = geoData.results[0]?.geometry?.location;
if (!coords) return new Response(JSON.stringify({ error: "Location not found" }

// Search for auto repair shops
const placesRes = await fetch(
  `https://maps.googleapis.com/maps/api/place/nearbysearch/json?location=${coord
  );
const placesData = await placesRes.json();
const places = placesData.results || [];

const leads = places.map((p: any) => ({
  type: "provider",
  name: p.name,
  location: p.vicinity,
  source: "continuous_discovery",
  status: "new",
  metadata: {
    place_id: p.place_id,
    rating: p.rating,
    user_ratings_total: p.user_ratings_total,
    open_now: p.opening_hours?.open_now
  }
})));

// Deduplication: filter out any leads whose name+location already exist in the
const filtered = [];
for (const lead of leads) {
  const { data: existingProfile } = await supabase
    .from("profiles")
    .select("id, role")
    .eq("business_name", lead.name)
    .maybeSingle();

  if (existingProfile) {
    // Already in CRM – upsert as linked/duplicate, do not send outreach
    filtered.push({ ...lead, crm_profile_id: existingProfile.id, crm_sync_statu
  } else {
    filtered.push({ ...lead, crm_sync_status: "unlinked" });
  }
}

```

```

if (filtered.length > 0) {
  const { error } = await supabase.from("outreach_leads").upsert(filtered, {
    onConflict: "name,location",
    ignoreDuplicates: true
  });
  if (error) return new Response(JSON.stringify({ error }), { status: 500 });
}

const newLeads = filtered.filter(l => l.crm_sync_status === "unlinked").length;
const duplicates = filtered.filter(l => l.crm_sync_status === "duplicate").length;

return new Response(JSON.stringify({ imported: newLeads, duplicates_skipped: duplicates,
  headers: { "Content-Type": "application/json" }
}));
});

```

REACT COMPONENT STRUCTURE

Create the following files inside your existing React admin portal. Suggested folder structure:

```

src/
  admin/
    outreach/
      OutreachDashboard.jsx      ← Main wrapper / navigation
      EngineControlPanel.jsx    ← Pause/Resume toggle + live engine status ←
      OpportunityPipeline.jsx   ← Live scrolling list of scored leads      ←
      OutreachHistoryPanel.jsx  ← Embeds inside existing CRM profile views ←
      LeadsTable.jsx            ← View and manage all leads
      AddLeadModal.jsx          ← Manual lead entry form
      ImportProvidersModal.jsx  ← Google Places import UI
      MessageQueue.jsx          ← Approve / reject drafted messages
      MessageComposer.jsx       ← Trigger AI draft for a lead
      CampaignManager.jsx       ← Create / manage campaigns
      OutreachAnalytics.jsx     ← Open rates, send counts, status breakdown
    hooks/
      useLeads.js
      useMessages.js
      useCampaigns.js
      useEngineState.js         ← Real-time engine state subscription ←
      usePipeline.js           ← Real-time pipeline feed subscription ←

```

COMPONENT SPECS

`OutreachDashboard.jsx`

Top-level layout. The `EngineControlPanel` is always visible at the top of every tab — it is a persistent header element, not hidden behind a tab. Below it, render tab-based navigation:

- **Pipeline** — live opportunity feed (default view)
 - **Queue** — messages drafted by AI awaiting admin approval
 - **Leads** — all leads with filtering by type and status
 - **Campaigns** — campaign list and creation
 - **Import** — Google Places import + CSV upload
 - **Analytics** — charts showing outreach volume, open rates, conversion by type
-

`EngineControlPanel.jsx` ← **CRITICAL: Build this first**

This is the persistent engine control bar rendered at the top of the admin outreach section. It must be visible at all times regardless of which tab is active.

Layout: A horizontal bar with the following elements left to right:

Left side — Engine Status Indicator:

- A pulsing green dot (animated CSS pulse) when running, gray dot when paused
- Label: "Engine Running" or "Engine Paused"
- Subtext showing last cycle time: *"Last cycle: 3 minutes ago"*

Center — Live Counters (auto-refreshing every 30s):

- Leads Discovered: 1,284
- Drafts in Queue: 17
- Messages Sent: 392

These pull from the `engine_state` table singleton row.

Right side — Pause/Resume Toggle:

- A large, clearly visible toggle switch

- When **running**: toggle is green, label reads "Engine Active" , clicking it opens a confirmation modal: *"Pause the outreach engine? The AI will stop discovering leads and drafting messages until you resume."* with a **Confirm Pause** button and optional reason text field
- When **paused**: toggle is red/gray, label reads "Engine Paused" , banner below shows: *"Paused [X minutes ago] by [admin]"* and optional reason — clicking resumes immediately with no confirmation required
- Also shows a settings gear icon that opens an `EngineSettingsModal` to configure: discovery interval, max drafts per cycle, and target cities

useEngineState.js hook: Subscribe to the `engine_state` table using Supabase Realtime so the panel updates live without polling:

```
import { useEffect, useState } from "react";
import { supabase } from "../../supabaseClient";

export function useEngineState() {
  const [engineState, setEngineState] = useState(null);

  useEffect(() => {
    // Initial fetch
    supabase.from("engine_state").select("*").eq("id", 1).single()
      .then(({ data }) => setEngineState(data));

    // Realtime subscription
    const channel = supabase
      .channel("engine-state-changes")
      .on("postgres_changes", {
        event: "UPDATE",
        schema: "public",
        table: "engine_state"
      }, (payload) => setEngineState(payload.new))
      .subscribe();

    return () => supabase.removeChannel(channel);
  }, []);

  const toggleEngine = async (is_running, pause_reason = null) => {
    await supabase.functions.invoke("toggle-engine", {
      body: { is_running, paused_by: "admin", pause_reason }
    });
  };

  return { engineState, toggleEngine };
}
```

}

OpportunityPipeline.jsx ← The live feed

This is the default landing tab of the outreach section. It shows a continuously-updating kanban-style or list view of all opportunities currently in the pipeline, sorted by score descending.

Layout: A filterable, sortable list (not paginated — use virtual scrolling for performance if list exceeds 200 rows) with the following columns:

Column	Details
Priority	Color-coded badge:  High,  Medium,  Low
Name	Lead name + type badge + CRM badge if <code>crm_sync_status = 'linked'</code> or <code>'converted'</code>
Score	0–100 score with a mini progress bar
AI Notes	The <code>ai_notes</code> field from the pipeline table
Channel	email / sms / both
Stage	Pill badge: new / draft_ready / message_queued / contacted / engaged / converted
CRM Status	 Linked (has matching profile),  Converted,  Re-engagement (crm source), — None
Added	Relative timestamp ("2 hours ago")
Actions	"Draft Now" button (hidden if <code>crm_sync_status = 'duplicate'</code>) → opens MessageComposer

Filter bar: Filter by `priority`, `stage`, and `type` (member / provider / investor)

Sort options: Score (default), Added Date, Priority

Realtime updates: Use Supabase Realtime to subscribe to inserts and updates on `opportunity_pipeline`. When a new lead is scored and added, it should animate into the list from the top with a subtle slide-in effect. Show a toast: *"3 new opportunities discovered"* when batch inserts occur.

usePipeline.js hook:

```
import { useEffect, useState } from "react";
import { supabase } from "../../supabaseClient";

export function usePipeline(filters = {}) {
  const [pipeline, setPipeline] = useState([]);
  const [newCount, setNewCount] = useState(0);

  useEffect(() => {
    const fetchPipeline = async () => {
      let query = supabase
        .from("opportunity_pipeline")
        .select("*, outreach_leads(*)")
        .order("opportunity_score", { ascending: false });

      if (filters.priority) query = query.eq("priority", filters.priority);
      if (filters.stage) query = query.eq("stage", filters.stage);

      const { data } = await query;
      setPipeline(data || []);
    };

    fetchPipeline();

    const channel = supabase
      .channel("pipeline-changes")
      .on("postgres_changes", {
        event: "*",
        schema: "public",
        table: "opportunity_pipeline"
      }, (payload) => {
        if (payload.eventType === "INSERT") {
          setNewCount(c => c + 1);
          setPipeline(prev => [payload.new, ...prev]);
        } else if (payload.eventType === "UPDATE") {
          setPipeline(prev =>
            prev.map(p => p.id === payload.new.id ? { ...p, ...payload.new } : p)
          );
        }
      })
      .subscribe();

    return () => supabase.removeChannel(channel);
  }, [JSON.stringify(filters)]);
```

```
    return { pipeline, newCount, clearNewCount: () => setNewCount(0) };  
  }  
}
```

MessageQueue.jsx

This is the most important component. It should:

1. Fetch all messages with `status = 'draft'` from `outreach_messages`, joined with lead info
 2. Display each message as a card showing:
 - Lead name, type badge (member / provider / investor), channel badge (email / SMS)
 - Subject line (if email)
 - Full message body in a readable preview (not truncated)
 - Two buttons: **Approve & Send** (green) and **Skip / Reject** (gray)
 - An optional **Edit Body** inline text area before approving
 3. When **Approve & Send** is clicked:
 - Update `status` to `'approved'` in Supabase
 - Immediately call the `send-outreach-message` Edge Function
 - Show a success toast notification
 4. Support bulk approval: a “Approve All” button at the top with a confirmation dialog
 5. Show an empty state illustration when queue is empty
-

MessageComposer.jsx

Triggered from the Leads table. Opens a modal that:

1. Shows the selected lead’s name, type, and contact info
2. Lets the admin choose: **Email** or **SMS** channel
3. Has a **“Draft with AI”** button that calls `draft-outreach-message` Edge Function and populates a text area with the result
4. Allows manual editing of the draft before saving

5. On save: inserts a new record into `outreach_messages` with `status = 'draft'`

6. Confirms: "Message added to approval queue."

`LeadsTable.jsx`

Full-featured data table:

- Columns: Name, Type, Channel, Email, Phone, Location, Source, Status, **CRM Link**, Created Date, Actions
 - The **CRM Link** column shows:
 - A clickable  `View in CRM` button for leads with `crm_profile_id` set — deep-links to the provider/member detail view in the existing admin portal
 -  `Converted` badge for `stage = converted`
 -  `Re-engagement` badge for `source = 'crm_reengagement'`
 -  `Duplicate` badge for `crm_sync_status = 'duplicate'` — Draft Message is disabled for these rows with a tooltip: *"This contact is already active in the CRM"*
 - Filter bar: filter by `type`, `status`, and `crm_sync_status`
 - Search box: search by name, email, or company
 - Action buttons per row:
 - **Draft Message** → opens `MessageComposer` (hidden for duplicates)
 - **Edit** → inline editing of notes/contact info
 - **Mark Converted** → updates status to `converted`, prompts to link to existing CRM profile by email lookup
 - **View in CRM** → opens the matching profile in the existing admin portal (if linked)
 - Bulk import button → opens `ImportProvidersModal`
 - Add Lead button → opens `AddLeadModal`
-

`AddLeadModal.jsx`

Form fields:

- Type (select: member / provider / investor) — required
- Name — required
- Email
- Phone
- Company
- Location / City
- Notes / Additional context

On submit: inserts into `outreach_leads` .

`ImportProvidersModal.jsx`

Two tabs:

Tab 1 — Google Places Import:

- City/address text input
- Radius selector (5 mi / 10 mi / 25 mi)
- “Search & Import” button → calls `import-provider-leads` Edge Function
- Shows results count and a preview list before confirming

Tab 2 — CSV Upload:

- Drag-and-drop CSV upload
 - Expected columns: `name, type, email, phone, company, location, notes`
 - Preview first 5 rows before importing
 - On confirm: bulk insert into `outreach_leads`
-

`OutreachAnalytics.jsx`

Display the following stats pulled from Supabase queries:

- **Cards (top row):** Total Leads | Messages Sent | Pending Approval | Conversions
- **Bar chart:** Messages sent per day (last 30 days)

- **Doughnut chart:** Lead breakdown by type (member / provider / investor)
- **Status funnel:** New → Contacted → Responded → Converted (horizontal bar)

Use Recharts for all charts (already included in most React setups).

COMPLIANCE & SAFETY GUARDRAILS

Build these directly into the components:

1. **Investor messages always require manual approval** — never allow auto-send for `type = 'investor'`. In the campaign settings, gray out the “Auto-send first touch” toggle for investor campaigns with a tooltip: *“Investor outreach requires manual review.”*
 2. **Unsubscribe tracking** — all email messages sent via Resend should include an unsubscribe footer. When a lead is marked `unsubscribed`, hide the “Draft Message” button and show a red badge. Never draft or send to unsubscribed leads.
 3. **Duplicate / existing CRM users are never contacted** — leads with `crm_sync_status = 'duplicate'` (already active members or providers in the platform) must never receive outreach messages. The Draft Message button must be hidden and the engine’s draft loop must skip these records entirely. Display a tooltip: *“This contact is already a platform user.”*
 4. **Re-engagement leads are flagged differently** — leads with `source = 'crm_reengagement'` are existing platform users who haven’t engaged. Their outreach messages should be softer and more personal (e.g., “We noticed you haven’t used MCC recently...”). The `draft-outreach-message` function should detect this source and adjust its Claude prompt tone accordingly.
 5. **Rate limiting UI warning** — if more than 50 messages are sent within a 24-hour period, show a yellow banner in the dashboard: *“High send volume detected. Monitor deliverability.”*
 6. **Duplicate protection** — before inserting a new lead, check if an email or phone already exists in both `outreach_leads` AND `profiles`. Show a warning if it does.
-

SUPABASE CRON JOBS (pg_cron)

Enable the `pg_cron` extension in Supabase and add the following scheduled jobs. The engine runs continuously on a short interval — the guard logic inside `engine-cycle` ensures

nothing executes when paused.

```
-- Enable pg_cron and pg_net (required for HTTP calls)
CREATE EXTENSION IF NOT EXISTS pg_cron;
CREATE EXTENSION IF NOT EXISTS pg_net;

-- PRIMARY ENGINE LOOP: Run every 15 minutes
-- The engine-cycle function checks is_running internally before doing any work
SELECT cron.schedule(
    'mcc-engine-cycle',
    '*/15 * * * *',
    $$
        SELECT net.http_post(
            url := current_setting('app.supabase_url') || '/functions/v1/engine-cycle',
            headers := jsonb_build_object(
                'Authorization', 'Bearer ' || current_setting('app.service_role_key'),
                'Content-Type', 'application/json'
            ),
            body := '{}'::jsonb
        );
    $$
);

-- FOLLOW-UP DRAFTS: Run daily at 9 AM – drafts follow-up messages for non-respon
SELECT cron.schedule(
    'mcc-followup-drafts',
    '0 9 * * *',
    $$
        SELECT net.http_post(
            url := current_setting('app.supabase_url') || '/functions/v1/draft-followup
            headers := jsonb_build_object(
                'Authorization', 'Bearer ' || current_setting('app.service_role_key'),
                'Content-Type', 'application/json'
            ),
            body := '{}'::jsonb
        );
    $$
);

-- PIPELINE CLEANUP: Run weekly – mark stale leads as 'dead' if no engagement in
SELECT cron.schedule(
    'mcc-pipeline-cleanup',
    '0 2 * * 0',
    $$
        UPDATE opportunity_pipeline
        SET stage = 'dead'
```

```

    WHERE stage IN ('new', 'draft_ready', 'message_queued')
    AND added_at < NOW() - INTERVAL '30 days';
$$
);

```

draft-followup-messages Edge Function

Create this additional Edge Function to handle the daily follow-up job. It respects engine pause state and drafts follow-up messages (sequence_step = 2) for leads contacted 3+ days ago with no reply:

```

// supabase/functions/draft-followup-messages/index.ts
import { serve } from "https://deno.land/std@0.168.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

serve(async (req) => {
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  // Guard: check engine state
  const { data: state } = await supabase.from("engine_state").select("is_running")
  if (!state?.is_running) {
    return new Response(JSON.stringify({ skipped: true, reason: "engine_paused" }
      headers: { "Content-Type": "application/json" }
    ));
  }

  // Find leads contacted 3+ days ago with no open/reply and only one message sen
  const { data: leads } = await supabase.rpc("get_followup_candidates");

  let drafted = 0;
  const baseUrl = Deno.env.get("SUPABASE_URL") + "/functions/v1";
  const headers = {
    "Authorization": `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")}`,
    "Content-Type": "application/json"
  };

  for (const lead of (leads || [])) {
    const draftRes = await fetch(`${baseUrl}/draft-outreach-message`, {
      method: "POST", headers,
      body: JSON.stringify({ lead, channel: lead.last_channel, sequence_step: 2 }
    ));
    const { subject, body } = await draftRes.json();
  }

```

```

    await supabase.from("outreach_messages").insert({
      lead_id: lead.id,
      channel: lead.last_channel || "email",
      sequence_step: 2,
      subject,
      body,
      status: "draft"
    });
    drafted++;
  }

  return new Response(JSON.stringify({ drafted }), {
    headers: { "Content-Type": "application/json" }
  });
});

```

Also add this helper SQL function to Supabase:

```

CREATE OR REPLACE FUNCTION get_followup_candidates()
RETURNS TABLE (id UUID, name TEXT, email TEXT, phone TEXT, type TEXT, last_channel TEXT)
LANGUAGE sql AS $$
  SELECT
    l.id, l.name, l.email, l.phone, l.type,
    m.channel AS last_channel
  FROM outreach_leads l
  JOIN outreach_messages m ON m.lead_id = l.id
  WHERE l.status = 'contacted'
    AND m.status = 'sent'
    AND m.sequence_step = 1
    AND m.opened_at IS NULL
    AND m.sent_at < NOW() - INTERVAL '3 days'
    AND NOT EXISTS (
      SELECT 1 FROM outreach_messages m2
      WHERE m2.lead_id = l.id AND m2.sequence_step >= 2
    )
  $$;

```

STYLING NOTES

- Match the existing MCC admin portal's design language
- Use color-coded badges: **blue** = member, **green** = provider, **gold/amber** = investor
- Status badges: gray = new, blue = queued, orange = contacted, green = converted, red

= unsubscribed

- Keep the approval queue front-and-center — it's the primary daily-use screen

TESTING CHECKLIST

After building, verify:

CRM Integration

- `OutreachHistoryPanel` renders correctly inside the existing Provider Application review view
- `OutreachHistoryPanel` renders correctly inside the existing Member Detail view
- Leads with matching `profiles` email are auto-detected and marked `crm_sync_status = 'duplicate'`
- Duplicate leads have Draft Message button disabled in LeadsTable and pipeline
- `auto_link_outreach_lead` trigger fires when a new profile is created and links matching outreach lead
- `convert-lead-to-profile` correctly updates both `outreach_leads` and `profiles` tables
- `sync-crm-reengagement` identifies dormant members and stalled provider applications correctly
- Re-engagement leads appear in the pipeline with `source = 'crm_reengagement'` badge
- "View in CRM" deep link in LeadsTable navigates to the correct admin profile view
- 🤖 Outreach Engine nav item appears in the existing admin sidebar in the correct position

Engine Control

- `EngineControlPanel` is visible on all tabs at all times
- Pause toggle opens a confirmation modal with optional reason field
- Confirming pause immediately updates `engine_state.is_running = false` in Supabase
- Engine status indicator changes to gray/paused state in real-time (no page refresh)
- Resume toggle reactivates without confirmation

- Paused-by and pause reason display correctly in the status bar
- `engine-cycle` Edge Function returns `{ skipped: true }` when engine is paused

Continuous Discovery

- `engine-cycle` runs and discovers new provider leads from Google Places for each target city
- New leads appear in `outreach_leads` table with `source = 'continuous_discovery'`
- `score-leads` correctly inserts rows into `opportunity_pipeline` with valid scores 0–100
- `OpportunityPipeline` feed updates in real-time as new leads are scored (no page refresh)
- Toast notification appears when batch of new leads is added

Message Drafting

- Lead can be added manually via form
- Google Places import returns results and inserts leads
- AI draft is generated correctly for all three lead types (member, provider, investor)
- Re-engagement drafts use a softer, personalized tone referencing prior platform activity
- Email sends successfully via Resend (check sandbox/test mode first)
- SMS sends successfully via Twilio (use a test number)
- Message status updates correctly in Supabase after send

Compliance Guardrails

- Investor messages cannot be auto-sent
- Unsubscribed leads are blocked from receiving new messages
- Duplicate (existing CRM user) leads cannot be messaged through any path
- Mid-cycle pause check halts draft loop immediately
- Analytics dashboard shows correct counts and updates after sends

Cron Jobs

- `mcc-engine-cycle` job appears in `cron.job` table in Supabase
- `mcc-followup-drafts` runs and respects pause state

- `mcc-pipeline-cleanup` marks stale entries correctly in test scenario

Build instructions prepared for My Car Concierge (MCC) AI Outreach Engine — Jordan Zanetis / Zanetis Holdings LLC